Suricata IDS Practical Lab

Beginner-Friendly • Authorized Cybersecurity Lab • GitHub Portfolio Ready

A beginner-friendly, authorized-lab exercise for learning how to deploy **Suricata as a network intrusion detection system (IDS)**, write a local detection rule, generate test traffic with Netcat, and analyze alerts in `fast.log` and `eve.json`.

***Authorization & safety:** Run this lab only on systems and networks you own or are explicitly authorized to test. The topology is intentionally isolated and the traffic-generation exercise is non-destructive.*

Learning outcomes

By completing this lab you will be able to:

• Identify the correct Linux network interface and IP address.

• Install and validate Suricata on Ubuntu/Debian.

• Configure `HOME_NET` and local rules.

• Write a basic Suricata TCP content rule.

• Generate controlled TCP traffic with Netcat.

• Start Suricata in IDS mode and verify packet capture.

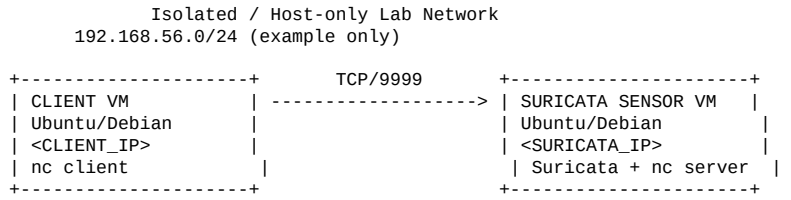
• Find and interpret alerts in `fast.log` and `eve.json`.

• Use `grep`, `jq`, `tail`, `ip`, and `ss` for SOC-style triage.

• Troubleshoot common connectivity and alerting problems.

• Document evidence in a GitHub-ready project.

Lab topology



The simplest design has the client send traffic directly to the sensor. This guarantees the sensor's interface sees the test traffic without requiring a network TAP or inline bridge.

Placeholders

| Placeholder | Meaning | Example |

|---|---|---|

| `` | IP address of the Suricata VM | `192.168.56.10` |

| `` | IP address of the client VM | `192.168.56.20` |

| `` | Suricata VM interface connected to the lab network | `enp0s8` |

| `` | Test TCP port | `9999` |

Replace placeholders before running commands.

Prerequisites

• Two Ubuntu/Debian VMs.

• An isolated host-only/internal network between them.

• Sudo access on the Suricata VM.

• Internet access temporarily available on the Suricata VM for package installation, or pre-staged packages.

• Basic Linux command-line knowledge.

• `nc`/Netcat on the client and sensor.

• `jq` on the Suricata VM.

• Do not expose the lab test service to an untrusted network.

Quick start

1. On the Suricata VM

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository ppa:oisf/suricata-stable
sudo apt update
sudo apt install -y suricata jq netcat-openbsd
sudo suricata --build-info
```

Expected: package installation completes and `suricata --build-info` prints version/build information.

2. Identify the interface

```
ip -br addr
ip route
```

Expected: identify `<INTERFACE>` and `<SURICATA_IP>` on the isolated lab network.

Example:

```
enp0s3          UP             10.0.2.15/24
enp0s8          UP             192.168.56.10/24
```

Here, `<INTERFACE>` is `enp0s8`.

3. Configure Suricata

Back up the configuration:

```
sudo cp /etc/suricata/suricata.yaml /etc/suricata/suricata.yaml.bak
```

Edit:

```
sudo nano /etc/suricata/suricata.yaml
```

Set `HOME_NET` to the sensor's lab IP/subnet. For this exercise:

```
vars:
  address-groups:
    HOME_NET: "[192.168.56.10]"
    EXTERNAL_NET: "any"
```

Make sure the rule file list includes:

```
rule-files:
  - suricata.rules
  - /etc/suricata/rules/local.rules
```

If your distribution's configuration already has a working `local.rules` entry, do not duplicate it.

Ensure these outputs are enabled:

```
outputs:
  - fast:
      enabled: yes
      filename: fast.log
      append: yes
```

```
  - eve-log:
      enabled: yes
      filetype: regular
      filename: eve.json
      types:
        - alert
        - flow
        - stats
```

The exact surrounding YAML can differ by Suricata version. Preserve indentation and existing settings.

4. Install the custom rule

```
sudo install -m 0644 rules/local.rules /etc/suricata/rules/local.rules
sudo suricata -T -c /etc/suricata/suricata.yaml
```

Expected ending:

```
Notice: Configuration provided was successfully loaded.
```

The exact wording varies by release. The important result is that the configuration test exits successfully with no fatal error.

5. Start a Netcat listener on the sensor

```
sudo nc -lvnp 9999
```

Expected:

```
Listening on 0.0.0.0 9999
```

Keep this terminal open.

6. Start Suricata on the sensor

Open a second terminal:

```
sudo suricata -c /etc/suricata/suricata.yaml -i <INTERFACE>
```

Expected: Suricata starts without a fatal error and remains attached to the terminal.

For a background service workflow, first stop any manually started Suricata process, then use:

```
sudo systemctl restart suricata
sudo systemctl status suricata --no-pager
```

Use one operating mode at a time to avoid two Suricata processes competing for the same interface/logs.

7. Generate test traffic from the client

```
printf 'SURICATA-LAB-TEST\n' | nc -nv -w 3 <SURICATA_IP> 9999
```

Expected on the client:

```
Connection to <SURICATA_IP> 9999 port [tcp/*] succeeded!
```

Expected on the sensor's Netcat terminal:

```
SURICATA-LAB-TEST
```

8. Check `fast.log`

On the sensor:

```
sudo tail -n 20 /var/log/suricata/fast.log
```

Expected: a line containing:

```
LOCAL LAB Netcat traffic to TCP/9999
```

9. Query EVE JSON

```
sudo jq 'select(.event_type=="alert") |
  {timestamp,src_ip,src_port,dest_ip,dest_port,proto,
   signature:.alert.signature,severity:.alert.severity}' \
  /var/log/suricata/eve.json | tail -n 20
```

Expected: structured JSON showing the client source IP, sensor destination IP, TCP/9999, and the custom signature.

Custom detection rule

File: `rules/local.rules`

```
alert tcp any any -> $HOME_NET 9999 (
    msg:"LOCAL LAB Netcat traffic to TCP/9999";
    flow:to_server,established;
    content:"SURICATA-LAB-TEST";
    nocase;
    sid:1000001;
    rev:1;
)
```

Rule breakdown

• `alert tcp` — generate an alert for matching TCP traffic.

• `any any` — accept any source IP and source port.

• `$HOME_NET 9999` — target the protected network on TCP port 9999.

• `flow:to_server,established` — inspect established client-to-server traffic.

• `content:"SURICATA-LAB-TEST"` — match the lab marker in the TCP payload.

• `nocase` — case-insensitive content matching.

• `sid:1000001` — local signature ID.

• `rev:1` — first revision.

Analyze logs with grep

Search the fast log:

```
sudo grep -i "LOCAL LAB Netcat" /var/log/suricata/fast.log
```

Expected: one or more alert lines matching the signature.

Search EVE JSON for the SID:

```
sudo grep '"signature_id":1000001' /var/log/suricata/eve.json
```

Expected: JSON alert records containing SID `1000001`.

Show only alert events:

```
sudo grep '"event_type":"alert"' /var/log/suricata/eve.json | tail -n 10
```

Analyze EVE JSON with jq

Pretty-print the latest alert:

```
sudo jq 'select(.event_type=="alert")' /var/log/suricata/eve.json | tail -n 1
```

Extract the most useful SOC fields:

```
sudo jq -r '
  select(.event_type=="alert") |
  [
    .timestamp,
    .src_ip,
    .src_port,
```

```
    .dest_ip,
    .dest_port,
    .proto,
    .alert.signature,
    .alert.severity,
    .alert.signature_id
  ] | @tsv
' /var/log/suricata/eve.json | tail -n 10
```

Expected format:

```
2026-10-07T...    <CLIENT_IP>    54321    <SURICATA_IP>    9999    TCP    LOCAL LAB Netcat traffic to TCP/9999    3    1000001
```

Count matching alerts:

```
sudo jq -r 'select(.event_type=="alert" and .alert.signature_id==1000001) | .alert.signature_id' \
  /var/log/suricata/eve.json | wc -l
```

Expected: a number greater than zero after the test traffic is generated.

Trigger and analyze alerts

1. Start the listener.

2. Start Suricata.

3. Confirm the sensor is listening:

```bash
sudo ss -lntp | grep ':9999'
```

4. From the client, send the marker:

```bash
printf 'SURICATA-LAB-TEST\n' | nc -nv -w 3 <SURICATA_IP> 9999
```

5. Confirm the listener receives the marker.

6. Check `fast.log`.

7. Check `eve.json`.

8. Record the source IP, destination IP, destination port, signature, SID, severity, and timestamp as evidence.

Troubleshooting

`No route to host`

Likely causes:

• VMs are not attached to the same host-only/internal network.

• Wrong IP address.

• Interface is down.

• Host firewall is rejecting traffic.

Checks:

```
ip -br addr
ip route
ping -c 3 <SURICATA_IP>
```

On the sensor:

```
sudo ip link set <INTERFACE> up
```

If using a host-only network, confirm both VMs use the same virtual network name/segment.

`Connection refused`

Likely cause: nothing is listening on TCP/9999.

Check:

```
sudo ss -lntp | grep ':9999'
```

Start the listener:

```
sudo nc -lvnp 9999
```

Then retry from the client.

Wrong interface

List interfaces:

```
ip -br addr
```

Check which interface owns `<SURICATA_IP>`:

```
ip -br addr | grep '<SURICATA_IP>'
```

Confirm packets are arriving:

```
sudo tcpdump -ni <INTERFACE> tcp port 9999
```

If the client connects but `tcpdump` shows nothing, Suricata is probably listening on the wrong interface or the traffic is not reaching that interface.

Missing alerts

Check the rule is installed:

```
sudo grep -n "LOCAL LAB Netcat" /etc/suricata/rules/local.rules
```

Validate configuration:

```
sudo suricata -T -c /etc/suricata/suricata.yaml
```

Confirm the rule file is referenced:

```
sudo grep -n "local.rules" /etc/suricata/suricata.yaml
```

Confirm the payload:

```
printf 'SURICATA-LAB-TEST\n' | nc -nv -w 3 <SURICATA_IP> 9999
```

Confirm packet visibility:

```
sudo tcpdump -ni <INTERFACE> -A tcp port 9999
```

Confirm Suricata is running once:

```
pgrep -a suricata
```

Avoid running both a manual foreground process and the systemd service simultaneously.

Verification checklist

• [ ] Two VMs are on an isolated lab network.

• [ ] `` and `` are correct.

• [ ] `` is the correct sensor interface.

• [ ] `suricata --build-info` works.

• [ ] `suricata -T` completes successfully.

• [ ] `local.rules` is loaded.

• [ ] Netcat listener is bound to TCP/9999.

• [ ] Client can connect to `:9999`.

• [ ] `tcpdump` can see the test traffic.

• [ ] `fast.log` contains the custom signature.

• [ ] `eve.json` contains an `event_type=alert` record.

• [ ] `jq` extracts the expected fields.

• [ ] Evidence is captured without exposing real credentials or sensitive data.

Student questions

1. Why must Suricata inspect the interface that actually receives the traffic?
2. What is the difference between `fast.log` and `eve.json`?
3. What does `$HOME_NET` represent?
4. Why does the rule use a unique SID?
5. What does `flow:to_server,established` accomplish?
6. What would happen if the marker string did not match the `content` keyword?
7. Why can a successful Netcat connection still produce no Suricata alert?
8. Which fields in EVE JSON would you use for a basic SOC triage record?

Practical challenge

Modify the lab so a second custom rule detects a different authorized test marker on TCP/9999.

Requirements:

• Use a new SID, e.g. `1000002`.

• Use a different message.

• Trigger it with a new Netcat payload.

• Validate the rule with `suricata -T`.

• Capture the alert in `fast.log`.

• Extract it from `eve.json` with `jq`.

• Add a short incident-style note under `evidence/challenge-result.md`.

Stretch goal: add a second rule that uses a different TCP destination port and explain why you chose that port.

GitHub evidence guidance

Good evidence:

• Sanitized terminal output.

• Sanitized `fast.log` alert.

• Sanitized EVE JSON alert.

• Network topology diagram.

• A short troubleshooting note.

Do not commit:

• Real passwords, API keys, private keys, tokens.

• Personal data.

• Unnecessary public IP addresses.

• Large raw logs.

• VM disk images.

Project structure

```
suricata-ids-practical-lab/
■■■ README.md
■■■ LICENSE
■■■ .gitignore
■■■ rules/
■   ■■■ local.rules
■■■ scripts/
■   ■■■ generate_traffic.sh
■   ■■■ start_listener.sh
■   ■■■ verify_lab.sh
■■■ examples/
■   ■■■ suricata.yaml.example
■■■ evidence/
■   ■■■ sample-fast.log
■   ■■■ sample-eve-alert.json
■   ■■■ challenge-result.md
■■■ docs/
■   ■■■ SURICATA_IDS_LAB_MANUAL.md
■   ■■■ COMMAND_CHEATSHEET.md
■■■ reports/
    ■■■ .gitkeep
```

References

• Suricata User Guide: https://docs.suricata.io/

• Suricata 8.0.7 documentation: https://docs.suricata.io/en/suricata-8.0.7/

• Suricata EVE JSON format: https://docs.suricata.io/en/suricata-8.0.7/output/eve/eve-json-format.html

• Suricata project: https://suricata.io/

Portfolio description

Built an isolated two-VM Suricata IDS lab to demonstrate network interface identification, IDS configuration, custom signature development, controlled traffic generation, alert triage, and EVE JSON analysis with grep/jq. Documented troubleshooting and verification steps in a reproducible GitHub project.